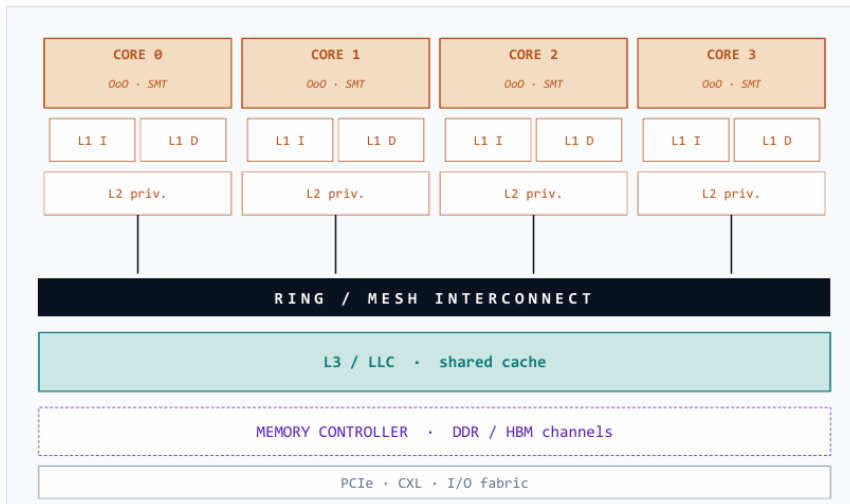


1. MIMD (Multiple instruction, multiple data)

1.2 Symmetric Multiprocessor (SMP)



In questo Sistema tutti i cori sono identici e supportano OoO (out of order) per minimizzare gli stalli della pipe e poi supporta il modello SMT (simultaneous multithreading). Ogni singolo core fisico può gestire contemporaneamente le istruzioni di più thread.

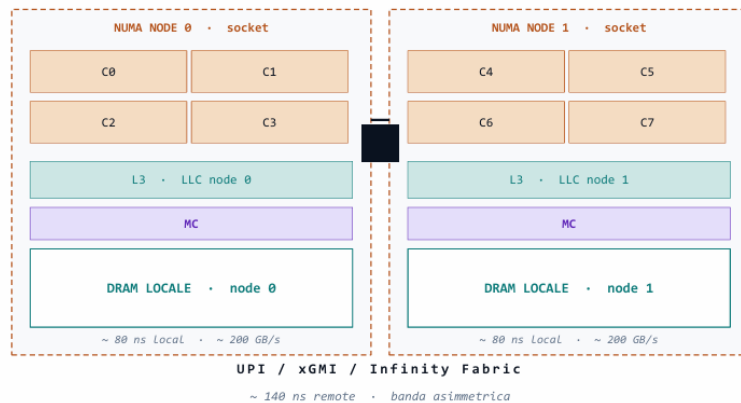
I core accedono alla stessa memoria principale con lo stesso tempo medio quindi modello UMA. Ogni core ha L1 e L2 privati per ridurre la pressione sullo strato di interconnessione. La cache L1 è divisa in cache istruzioni e cache dati per evitare conflitti. La cache L3 è condivisa ed è il punto di sincronizzazione tra i core.

La rete di interconnessione non è un semplice bus che scalerebbe poco, ma si usano topologie ad anello o a griglia. Attraverso questo si accede alla cache L3

Abbiamo un problema di coerenza della cache che è garantita grazie alla tecnica di snooping in hardware. Il programmatore non deve preoccuparsi di propagare le scritture: usa primitive di sincronizzazione di alto livello come mutex per garantire la consistenza ovvero l'ordine logico di accesso.

Il limite di quest'architettura è il collo di bottiglia del sistema di interconnessione. Oltre i 16-64 core per dire satura e si è costretti a passare al multi-socket NUMA.

1.2 NUMA



Il limite dei core nei SMP porta all'utilizzo del modello di memoria NUMA.

Nei sistemi SMP hai un solo processore fisico. Questo processore si inserisce in uno slot sulla scheda madre: il socket. Tutti i core quindi condividono lo stesso accesso alla memoria RAM.

Quando un singolo blocco di silicio non può più fisicamente ospitare più core, la soluzione è moltiplicare i processori fisici sulla stessa scheda madre. Abbiamo ad esempio 2,4,8 socket.

Ogni processore fisico (socket) ha il suo memory controller e le sue memorie con i livelli di cache come in SMP. Accedere alla propria memoria è veloce, ma se il Core 0 del socket A ha bisogno di un dato nella memoria gestita dal Socket B, la richiesta deve viaggiare su un bus esterno pagando una penalità di latenza.

Quando un programma C/C++ richiede memoria tramite malloc ad esempio, il sistema operativo non alloca neanche un byte di RAM fisica.

Si limita solo ad aggiornare le strutture dati interne del processo, assegnare un intervallo di indirizzi virtuali e lasciare la page table Entries vuote. L'array che volevamo allocare esiste solo sulla carta.

Il programma prosegue e un thread esegue la primissima istruzione di scrittura o lettura su uno degli indirizzi virtuali:

- La CPU invia l'indirizzo virtuale all'MMU
- La MMU consulta il TLB e la page table scoprendo che quell'indirizzo virtuale non corrisponde a nessun indirizzo fisico
- Page fault

Linux adotta l'euristica più logica ovvero Quale core della CPU ha appena generato questo page fault?

Se il page Fault è stato innescato dal core 4 del socket 1, il kernel deduce che il core 4 lavorerà su quei dati. Quindi dice al Memory Controller del Nodo 1 di riservare una pagina fisica nella sua DRAM.

L'euristica del first touch è ottima per i programmi a singolo thread perché garantisce che la memoria fisica si trovi il più vicino possibile alla CPU che la userà.

Nel calcolo parallelo però crea un disastro. Se scriviamo un codice in cui il thread principale (Che si trova nel Nodo 0) inizializza tutto l'array con un ciclo quindi fa operazioni di scrittura, prima di lanciare i thread worker, sarà il Nodo 0 a generare i page fault consecutivi.

Il kernel piazzerà l'intero array nella RAM del Nodo 0. Quando poi i worker che stanno nel Nodo 1 cercheranno di elaborare i dati, la latenza aumenterà a dismisura perché bisogna passare per i collegamenti tra un socket e un altro.

Quindi inizializzare un array nel thread sbagliato condanna tutti gli accessi successivi.

Linux utilizza la tecnica del pinning: significa dare un ordine tassativo al sistema operativo: *"Il Thread 0 deve girare SOLO sul Core 0, il Thread 1 SOLO sul Core 1, il Thread 4 SOLO sul Core 4"*. I thread diventano immobili, ancorati alla loro posizione geografica nell'hardware.